

# 浙江大学

## 本科实验报告

|       |                            |
|-------|----------------------------|
| 课程名称: | 课程综合实践 I-<br>深度学习基础——理论与实践 |
| 姓 名:  | 李昊泽                        |
| 学 院:  | 计算机科学与技术学院                 |
| 系:    |                            |
| 专 业:  | 计算机科学与技术                   |
| 学 号:  | 3250102780                 |
| 指导教师: | 王总辉                        |

2026 年 9 月 1 日

# 浙江大学实验报告

课程名称: 课程综合实践 I- 深度学习基础——理论与实践 实验类型: 综合

实验项目名称: 实验二: 全连接神经网络

学生姓名: 李昊泽 专业: 计算机科学与技术 学号: 3250102780

实验地点: 紫金港海洋大楼 11 机房 实验日期: 2026 年 9 月 1 日 指导老师: 王总辉

## 摘要

本实验围绕全连接神经网络完成两个分类任务。第一部分在 Kaggle 平台使用 Adult Census Income 数据, 根据个人基本信息预测年收入是否超过 50K。使用结构为  $106 \rightarrow 64 \rightarrow 64 \rightarrow 2$  的全连接神经网络完成二分类, 并按验证损失保存最佳模型、生成竞赛提交文件。

第二部分在 LMCC 实验环境中使用美国手语图像数据, 把每张  $28 \times 28$  灰度图像展平为 784 维向量, 使用  $784 \rightarrow 512 \rightarrow 512 \rightarrow 26$  的多层感知机进行训练和验证。

两个任务都使用 ReLU、交叉熵损失和 Adam 优化器, 但输入规模、类别数量及泛化表现不同。Adult Income 模型在本地复现中取得 85.46% 的最佳验证准确率; ASL 模型最终达到 100.00% 的训练准确率和 82.31% 的验证准确率, 截图中可见的最高验证准确率为 82.35%, 训练集与验证集之间仍存在明显差距。实验说明了前向传播、反向传播、验证与模型选择的完整流程, 也展示了仅提高训练准确率并不等于提高模型泛化能力。所以我们要引入下一节课的 CNN 来通过卷积体现二维的空间结构。

## 实验一: Kaggle 平台个人年收入预测

### 一、任务要求

本任务根据年龄、工作类别、学历、婚姻状况、职业、资本收益、每周工作时长等个人信息, 判断年收入是否超过 50K。标签 0 表示收入小于等于 50K, 标签 1 表示收入大于 50K, 因此它属于二分类问题, 而不是像 PM2.5 实验那样预测连续

数值的回归问题。

| 文件      | 样本数   | 特征/标签数  | 作用        |
|---------|-------|---------|-----------|
| X_train | 32561 | 106 个特征 | 模型训练与验证输入 |
| Y_train | 32561 | 1 个标签   | 收入二分类监督信号 |
| X_test  | 16281 | 106 个特征 | 生成竞赛预测    |

**Tip**

训练标签分布为 0 类 24720 个、1 类 7841 个，可见类别并不完全均衡，因此划分训练集与验证集时采用分层抽样。

```
X_train: /kaggle/input/competitions/2026zjutest2/X_train
Y_train: /kaggle/input/competitions/2026zjutest2/Y_train
X_test : /kaggle/input/competitions/2026zjutest2/X_test
运行设备: cpu
```

Figure 1: 数据加载

```
X_train: (32561, 106)
Y_train: (32561,)
X_test : (16281, 106)
实际输入维度: 106
标签分布: {np.int64(0): np.int64(24720), np.int64(1): np.int64(7841)}
```

|   | age | fnlwgt | sex | capital_gain | capital_loss | hours_per_week | Federal-gov | Local-gov | Never-worked | Private | .. |
|---|-----|--------|-----|--------------|--------------|----------------|-------------|-----------|--------------|---------|----|
| 0 | 39  | 77516  | 1   | 2174         | 0            | 40             | 0           | 0         | 0            | 0       | .. |
| 1 | 50  | 83311  | 1   | 0            | 0            | 13             | 0           | 0         | 0            | 0       | .. |
| 2 | 38  | 215646 | 1   | 0            | 0            | 40             | 0           | 0         | 0            | 1       | .. |
| 3 | 53  | 234721 | 1   | 0            | 0            | 40             | 0           | 0         | 0            | 1       | .. |
| 4 | 28  | 338409 | 0   | 0            | 0            | 40             | 0           | 0         | 0            | 1       | .. |

5 rows x 106 columns

Figure 2: 形状检查

## 二、数据预处理

读取数据后，代码先检查训练和测试特征的列名与顺序是否一致，再把无法转换的内容、正负无穷和缺失值统一处理为 0。训练集按标签进行 8:2 分层划分，得到 26049 个训练样本和 6512 个验证样本。

各特征的数值范围差异较大，因此按列进行标准化。设训练子集第  $j$  个特征的均值和标准差分别为  $\mu_j$  和  $\sigma_j$ ，则：

$$x'_j = \frac{x_j - \mu_j}{\sigma_j}$$

均值和标准差只在训练子集上计算，再同时应用于训练、验证和测试数据。这样既保证三部分数据位于同一尺度，也避免利用验证集统计量造成信息泄漏。当某列标准差接近 0 时，将其标准差替换为 1，防止除零产生 NaN。

```

1 feature_mean = X_all[train_idx].mean(axis=0)
2 feature_std = X_all[train_idx].std(axis=0)
3 feature_std[feature_std < 1e-8] = 1.0
4
5 X_train = (X_all[train_idx] - feature_mean) / feature_std
6 X_valid = (X_all[valid_idx] - feature_mean) / feature_std
7 X_test = (X_test_all - feature_mean) / feature_std

```

### 三、网络结构与工作原理

本实验使用一个三层全连接网络。网络输出两个原始分数，即两个类别的 logits：

| 层             | 输入与输出     | 作用                          |
|---------------|-----------|-----------------------------|
| Flatten       | 106 → 106 | 保持批次维度，将单个样本整理成一维向量         |
| Linear + ReLU | 106 → 64  | 学习个人属性的非线性组合                |
| Linear + ReLU | 64 → 64   | 进一步组合中间特征                   |
| Linear        | 64 → 2    | 输出“≤50K”和“>50K”两个类别的 logits |

前向传播可以写成：

$$\begin{aligned}
 h_1 &= \text{ReLU}(W_1x + b_1) \\
 h_2 &= \text{ReLU}(W_2h_1 + b_2) \\
 z &= W_3h_2 + b_3
 \end{aligned}$$

其中 `ReLU(x)=max(0,x)` 为网络引入非线性。如果隐藏层之间没有激活函数，多层线性变换仍可合并为一个线性变换，网络就难以学习复杂的分类边界。本网络共有 11138 个可训练参数。输出层不显式添加 Sigmoid 或 Softmax，因为 `CrossEntropyLoss` 直接接收 logits，并在内部以数值更稳定的方式完成相应计算；预测时使用 `argmax` 选择分数最大的类别。

```

1 model = nn.Sequential(
2     nn.Flatten(),
3     nn.Linear(INPUT_SIZE, 64), nn.ReLU(),
4     nn.Linear(64, 64), nn.ReLU(),
5     nn.Linear(64, 2),
6 )

```

### 四、训练、验证与最佳模型选择

模型使用批大小 256、交叉熵损失和 Adam 优化器，学习率为 0.001，共训练 20 个 epoch。每个批次依次执行前向传播、计算损失、清空旧梯度、反向传播和参数更新：

```

1 optimizer.zero_grad(set_to_none=True)
2 logits = model(features)

```

```

3 loss = criterion(logits, labels)
4 loss.backward()
5 optimizer.step()

```

反向传播根据链式法则计算损失对各层权重和偏置的梯度；Adam 再结合梯度的一阶矩和二阶矩估计，自适应地调整不同参数的更新步长。验证阶段使用 `model.eval()` 和 `torch.no_grad()`，不执行 `backward()` 与 `optimizer.step()`，因此验证数据不会参与参数更新。

每个 epoch 结束后比较验证损失。如果当前验证损失低于历史最小值，就保存模型的 `state_dict` 到 `MyModels/adult_Model.pth`。只保存纯权重可以与 PyTorch 2.6 的 `weights_only=True` 安全加载机制兼容。

## 五、实验结果与分析

固定随机种子 42 后，本地完整复现的代表性结果如下：

| Epoch | 训练 Loss | 训练 Acc | 验证 Loss | 验证 Acc |
|-------|---------|--------|---------|--------|
| 1     | 0.4168  | 80.09% | 0.3358  | 84.32% |
| 5     | 0.3019  | 86.04% | 0.3138  | 85.46% |
| 10    | 0.2863  | 86.71% | 0.3201  | 85.12% |
| 20    | 0.2672  | 87.60% | 0.3376  | 84.83% |

```

Epoch 01/20 | train loss 0.4168, acc 0.8009 | valid loss 0.3358, acc 0.8432
Epoch 02/20 | train loss 0.3282, acc 0.8463 | valid loss 0.3206, acc 0.8504
Epoch 03/20 | train loss 0.3149, acc 0.8541 | valid loss 0.3168, acc 0.8490
Epoch 04/20 | train loss 0.3071, acc 0.8572 | valid loss 0.3168, acc 0.8533
Epoch 05/20 | train loss 0.3019, acc 0.8604 | valid loss 0.3138, acc 0.8546
Epoch 06/20 | train loss 0.2978, acc 0.8612 | valid loss 0.3157, acc 0.8557
Epoch 07/20 | train loss 0.2945, acc 0.8624 | valid loss 0.3168, acc 0.8514
Epoch 08/20 | train loss 0.2914, acc 0.8650 | valid loss 0.3200, acc 0.8546
Epoch 09/20 | train loss 0.2894, acc 0.8656 | valid loss 0.3174, acc 0.8524
Epoch 10/20 | train loss 0.2863, acc 0.8671 | valid loss 0.3201, acc 0.8512
Epoch 11/20 | train loss 0.2837, acc 0.8689 | valid loss 0.3252, acc 0.8509
Epoch 12/20 | train loss 0.2831, acc 0.8682 | valid loss 0.3186, acc 0.8529
Epoch 13/20 | train loss 0.2804, acc 0.8705 | valid loss 0.3256, acc 0.8507
Epoch 14/20 | train loss 0.2791, acc 0.8720 | valid loss 0.3237, acc 0.8507
Epoch 15/20 | train loss 0.2762, acc 0.8720 | valid loss 0.3274, acc 0.8481
Epoch 16/20 | train loss 0.2743, acc 0.8729 | valid loss 0.3328, acc 0.8492
Epoch 17/20 | train loss 0.2733, acc 0.8740 | valid loss 0.3318, acc 0.8523
Epoch 18/20 | train loss 0.2717, acc 0.8749 | valid loss 0.3354, acc 0.8449
Epoch 19/20 | train loss 0.2711, acc 0.8745 | valid loss 0.3361, acc 0.8480
Epoch 20/20 | train loss 0.2672, acc 0.8760 | valid loss 0.3376, acc 0.8483
最佳模型: epoch 5, 验证 loss 0.3138
模型保存到: /kaggle/working/MyModels/adult_Model.pth

```

Figure 3: 训练结果

第 5 轮获得最小验证损失 0.3138，同时验证准确率为 85.46%，因此被保存为最佳模型。此后训练损失继续下降、训练准确率继续上升，但验证损失逐步增大，说明模型开始轻微过拟合。

## 实验二：LMCC 平台美国手语图像识别

### 一、任务要求

本任务使用 Sign Language MNIST 数据集识别美国手语字母。每张图片为  $28 \times 28$  的单通道灰度图像，每行 CSV 包含一个 `label` 和 784 个像素值。J 和 Z 需要运动过程才能表达，因此静态图片数据集中不包含这两个字母。

本地资料中的 `sign_mnist_train.csv` 为 27455 行、785 列，`sign_mnist_valid.csv` 为 7172 行、785 列；两者的像素范围均为 0 至 255。实际检查发现标签已经重映射为连续的 0 至 23，共 24 类。

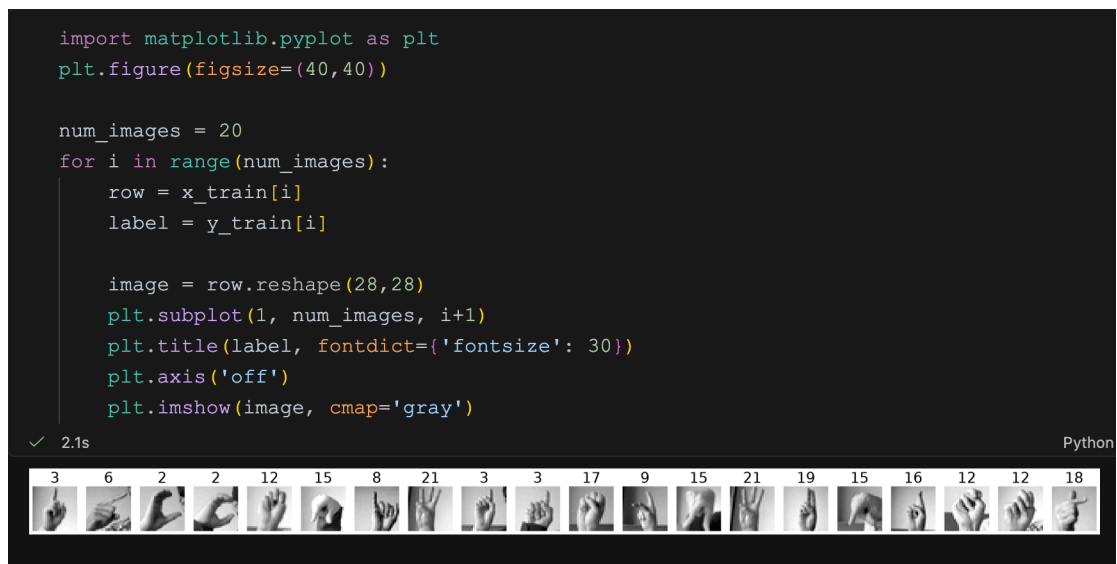
| 数据集 | 样本数   | 像素数 | 标签            |
|-----|-------|-----|---------------|
| 训练集 | 27455 | 784 | 0 ~ 23，共 24 类 |
| 验证集 | 7172  | 784 | 0 ~ 23，共 24 类 |

### 二、数据预处理

使用 Pandas 读取 CSV 后，通过 `pop("label")` 从 DataFrame 中分离标签，剩余 784 列转换为图像特征。为了显示图片，需要把一维像素向量重新变形为  $28 \times 28$ ；为了训练，则把像素除以 255，将数值范围从 0 ~ 255 映射为 0 ~ 1。

```
... 0      3
    1      6
    2      2
    3      2
    4     12
    ..
27450    12
27451    22
27452    17
27453    16
27454    22
Name: label, Length: 27455, dtype: int64
```

```
1 y_train = train_df.pop("label")
2 y_valid = valid_df.pop("label")
3 x_train = train_df.values / 255
4 x_valid = valid_df.values / 255
5
6 image = x_train[0].reshape(28, 28)
```



归一化不会改变图像内容，只改变数值尺度。统一而较小的输入范围可减小梯度尺度差异，使 Adam 的更新更加稳定。

### 三、Dataset 与 DataLoader

定义 `MyDataset`，在初始化时把特征转换为浮点张量、标签转换为整型张量，并将二者移动到运行设备。`DataLoader` 再按照批大小 32 产生小批量数据。训练集使用 `shuffle=True`，使每个 epoch 的样本顺序不同；验证集不打乱，便于稳定复现评价过程。

```
1 class MyDataset(Dataset):
2     def __init__(self, x_df, y_df):
3         self.xs = torch.tensor(x_df).float().to(device)
4         self.ys = torch.tensor(y_df).to(device)
5
6 train_loader = DataLoader(train_data, batch_size=32, shuffle=True)
7 valid_loader = DataLoader(valid_data, batch_size=32)
```

Python

每个训练批次的特征形状为 `(32,784)`，标签形状为 `(32,)`。

### 四、网络结构

网络结构如下：

| 层             | 输入与输出       | 作用                      |
|---------------|-------------|-------------------------|
| Flatten       | 28×28 → 784 | 将二维灰度图像展开为一维像素向量        |
| Linear + ReLU | 784 → 512   | 从全局像素组合中提取初级手势特征        |
| Linear + ReLU | 512 → 512   | 把初级特征组合为更复杂的手掌与手指模式     |
| Linear        | 512 → 26    | 输出课程代码设定的 26 个类别 logits |

由交叉熵损失直接处理 logits:

$$h_1 = \text{ReLU}(W_1x + b_1), \quad h_2 = \text{ReLU}(W_2h_1 + b_2) \\ z = W_3h_2 + b_3$$

```
Sequential(
  (0): Flatten(start_dim=1, end_dim=-1)
  (1): Linear(in_features=784, out_features=512, bias=True)
  (2): ReLU()
  (3): Linear(in_features=512, out_features=512, bias=True)
  (4): ReLU()
  (5): Linear(in_features=512, out_features=26, bias=True)
)
```

## 五、损失函数、反向传播与验证

交叉熵损失衡量模型对真实类别分配的相对置信度。对一个批次而言，可写为：

$$L = -\frac{1}{N} \sum_{i=1}^N \log p_{i,y_i}$$

当模型给真实类别的概率越低，损失越大。`loss.backward()` 根据链式法则求出损失对所有权重和偏置的梯度，`optimizer.step()` 再由 Adam 更新参数。训练模式下每个批次均执行更新；验证阶段置为 `model.eval()`，并在 `torch.no_grad()` 中只进行前向传播。

## 六、训练结果与分析

使用批大小 32 和 Adam 默认参数训练 20 个 epoch。图中 epoch 从 0 开始编号，因此最后一轮为 Epoch 19。程序输出的 Loss 是每个批次平均损失的累计值，没有再除以批次数；它适合观察同一数据集在不同 epoch 间的变化，但训练 Loss 与验证 Loss 不宜直接按绝对数值比较。



```
Epoch: 0
Train - Loss: 1515.1573 Accuracy: 0.4219
Valid - Loss: 309.0638 Accuracy: 0.5318
Epoch: 1
Train - Loss: 683.7674 Accuracy: 0.7298
Valid - Loss: 258.8107 Accuracy: 0.6560
Epoch: 2
Train - Loss: 349.8291 Accuracy: 0.8669
Valid - Loss: 218.9519 Accuracy: 0.7280
Epoch: 3
Train - Loss: 205.2343 Accuracy: 0.9244
Valid - Loss: 233.4721 Accuracy: 0.7182
Epoch: 4
Train - Loss: 114.8464 Accuracy: 0.9607
Valid - Loss: 217.9326 Accuracy: 0.7599
Epoch: 5
Train - Loss: 92.2633 Accuracy: 0.9669
Valid - Loss: 219.0865 Accuracy: 0.7656
Epoch: 6
Train - Loss: 57.1494 Accuracy: 0.9805
Valid - Loss: 234.0733 Accuracy: 0.7658
Epoch: 7
Train - Loss: 55.1908 Accuracy: 0.9807
Valid - Loss: 223.9549 Accuracy: 0.8084
Epoch: 8
...
Valid - Loss: 226.7476 Accuracy: 0.8235
Epoch: 19
Train - Loss: 0.4768 Accuracy: 1.0000
Valid - Loss: 232.7218 Accuracy: 0.8231
```

Figure 7: ASL 20 轮训练结果

| Epoch | 训练 Loss (累计) | 训练 Acc  | 验证 Loss (累计) | 验证 Acc |
|-------|--------------|---------|--------------|--------|
| 0     | 1515.1573    | 42.19%  | 309.0638     | 53.18% |
| 1     | 683.7674     | 72.98%  | 258.8107     | 65.60% |
| 2     | 349.8291     | 86.69%  | 218.9519     | 72.80% |
| 4     | 114.8464     | 96.07%  | 217.9326     | 75.99% |
| 7     | 55.1908      | 98.07%  | 223.9549     | 80.84% |
| 19    | 0.4768       | 100.00% | 232.7218     | 82.31% |

训练过程中，训练准确率总体由 Epoch 0 的 42.19% 提高，在 Epoch 19 达到 100.00%；验证准确率由 53.18% 提升到最终的 82.31%，截图省略部分后显示的一次验证结果达到 82.35%。最后一轮训练准确率比验证准确率高 17.69 个百分点，且可见的验证累计 Loss 在 Epoch 4 降至 217.9326 后，最终回升至 232.7218。这说明模型已经充分拟合训练数据，但对未参与训练的数据仍有明显的泛化差距，表现出过拟合。

过拟合的原因主要包括：全连接网络约有 67.8 万个参数，容量相对于数据量较大；把图像展平后削弱了二维空间结构；模型没有 Dropout、权重衰减和数据增强；训练 20 轮但没有按验证指标保存最佳模型。

#### Tip

可尝试采用以下改进：

1. 保存验证损失最低或验证准确率最高的模型，而不是直接使用最后一轮。
2. 在隐藏层加入 Dropout，或在 Adam 中设置 `weight_decay`。
3. 对训练图像进行轻微平移、旋转等数据增强。
4. 使用卷积神经网络保留局部空间关系，更有效地学习手指边缘与轮廓。
5. 将输出类别数与当前数据的 24 个标签保持一致。

## 实验总结

神经网络训练不是简单地把训练准确率提高到最大。损失函数描述错误，反向传播计算参数调整方向，优化器执行更新，而验证集负责检验这些更新能否推广到未参与训练的数据。当训练指标继续改善、验证指标反而恶化时，应识别为过拟合，并使用最佳模型保存、正则化、数据增强或更适合数据结构的网络来改善泛化能力。

## 参考资料

1. 实验说明，课程实验资料。
2. 本报告的部分文字、代码核对与排错建议由 AI 辅助生成。所有数据形状、运行结果和技术描述均经过本人检查；最终报告由本人审阅并负责。